EasyMap: Improving Technology Mapping via Exploration-Enhanced Heuristics and Adaptive Sequencing

Peiyu Wang[1†], Anqi Lu[1†], Xing Li[2], Junjie Ye[2], Lei Chen[2], Mingxuan Yuan[2], Jianye Hao[2], Junchi Yan[1*]
[1] Department of CSE & MoE Key Lab of AI, Shanghai Jiao Tong University, Shanghai, China
[2] Huawei Noah's ark Lab, Shenzhen & Hong Kong & Beijing, China
{wwwpy123, anqi_lu, yanjunchi}@sjtu.edu.cn
{li.xing2, yejunjie4, lc.leichen, yuan.mingxuan, haojianye}@huawei.com

2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD) | 979-8-3503-2225-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCAD57390.2023.10323703

***Abstract*—Technology mapping is a crucial step in the logic synthesis in chip design e.g. Field Programmable Gate Arrays (FPGAs) design, where a logic network is transformed into a K-bounded lookup tables (K-LUTs) network. Traditional mapping algorithms converges quickly to a suboptimal result, which limits the exploration capacity for further improvement. In this paper, we propose a new mapping method called Exploration-enhanced heuristics and Adaptive sequencing for Technology Mapping (EasyMap). EasyMap includes a pool of new heuristics and considers the mapping exploration as a conditional sequence optimization problem. During the mapping exploration procedure, heuristic algorithms with specific parameters are selected and applied sequentially. Our EasyMap outperforms the widely used IfMap in ABC by a significant margin. In particular, when optimizing area with a level constraint, EasyMap outperforms IfMap by reducing 9.1% more area on arithmetic circuits of the EPFL benchmark. Moreover, when optimizing area without level constraints at the same time, EasyMap can reduce 19% more area than IfMap on arithmetic circuits.**

***Index Terms*—FPGA design, Technology Mapping, Exploration-enhanced Heuristics, Adaptive Sequencing, Area Reduction, Black-box Optimization**

I. Introduction

Look-up Table (LUT) mapping or more broadly speaking, technology mapping is an essential step in the synthesis of Field Programmable Gate Arrays (FPGA) design. It transforms the And-Inverted Graph (AIG) into a functionally equivalent netlist comprising LUTs, whose quality could heavily influence the FPGA performance w.r.t. certain metrics such as area, delay, edge and power [1]. Over the last few decades, significant progress has been made in mapping optimization for FPGA design synthesis [2]–[4].

Among the above different metrics, level and area have been the central goal in technology mapping. The level count directly influences the signal delay and smaller area would save both space and power cost.

Specifically, the LUT-based technology mapping [5]–[7] problem can be viewed as a graph partition or matching task [8], which is computationally tractable in polynomial time when the target is for level number reduction optimization e.g. by FlowMap [2], but is NP-hard for area optimization [9].

There have been a number of traditional methods for level or/and area optimization. Improving upon FlowMap, CutMap [10] is developed to additionally optimize area while maintaining optimal delay. Both FlowMap and CutMap identify "good" cuts using the maximum-flow algorithm to achieve level count reduction, but at the expense of increased runtime complexity and hurting the area performance. DAOMap [3] applies dynamic programming to optimize delay and heuristics to improve area based on K-feasible cuts enumeration. However, the memory and runtime can still be huge due to the exponent complexity of cuts numbers, which is $O(n^k)$ for AIG with n nodes. To overcome this issue, priority cuts [4] are introduced to generate only a small, fixed number of "good" K-feasible cuts at each node, achieving high-quality area optimization while minimizing memory and runtime overheads.

With the widespread application of machine learning in EDA, AI for logic synthesis has also emerged as a topic worthy of attention [11]–[13]. [14] is the first work to apply machine learning to technology mapping. It performs cut-embedding by extracting graph representations and trains using Convolutional Neural Network (CNN) on its randomly generated training set, achieving excellent results.

In particular, in the widely used open-source synthesis tool ABC [15], the mapping engine, called IfMap, utilizes priority cuts to optimize level and area, achieving excellent performance with efficient runtime and memory cost. It calculates a utility value for each cut and updates the value iteratively. During mapping, the cuts with small utility values are given priority. However, when optimizing for area, IfMap often gets stuck at "local" optima after a few iterations, limiting its space for further optimization. Fig. 1 illustrates this behavior, where the area curve drops and converges after few iterations on circuits of the EPFL benchmark [16].

In this paper, we take a deep dive into the above unwanted fast convergence behavior of the widely used IfMap method and accordingly propose a set of exploration-enhanced heuristics, which expand the number of algorithms in the processing

Peiyu Wang and Anqi Lu contribute equally. Junchi Yan is the correspondence author. SJTU authors were supported by China Research and Development Program (2020AAA0107600), NSFC (62222607), Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102).

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

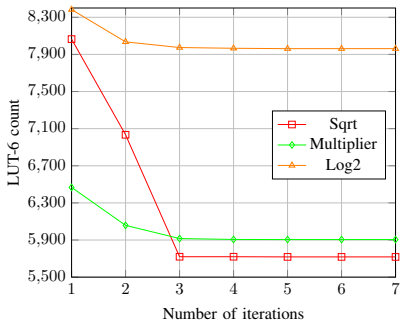

Fig. 1. Fast-convergence of IfMap on Sqrt, Multiplier, Log2 circuits of EPFL benchmark. Increasing the number of iterations for Global Area Flow will eventually lead to a convergent solution similar to the results in the table.

sequence from two in existing methods to over 10, which not only significantly expands the processing capacity, but also induces the opportunity to introduce heuristic running schedule optimization to boost the overall performance in a way similar to [17]–[20] as used in . We call our new mapping method Exploration-enhanced heuristics and Adaptive sequencing for Technology Mapping (EasyMap), which outperforms IfMap by a significant margin as will be shown in our experiments. The main contributions of this work are as follows:

- We conduct a formal theoretical analysis of IfMap's convergence behavior which to our best knowledge is new in literature, and we further propose two new heuristic algorithms based on our analysis.
- We design new utility functions that can capture different features of cuts and develop new heuristic approaches by combining them with different heuristic algorithms.
- We offer generic heuristics running sequences and use black-box optimization to get customized sequences for each circuits, to further boost the performance of SOTA.
- Our new mapping method, EasyMap, achieves significant improvements comparing with IfMap. In particular, when both optimizing area with/without the level constraint, EasyMap outperforms IfMap by reducing 9.1%/19% more area on arithmetic circuits of the EPFL benchmark.

II. Background

This section provides necessary notations and related backgrounds for Cut-based Mapping and IfMap.

Cut-based Technology Mapping. An *AIG* is a directed acyclic graph (DAG), denoted as $G = (N, E)$, where nodes represent logic gates AND and constants (0 or 1). In an AIG, nodes without incoming edges (also known as *fanin*) are called *Primary Inputs (PIs)*, while nodes without outgoing edges (also known as *fanout*) are called *Primary Outputs (POs)*. In addition to these two types of nodes, each node in the network has two incoming edges and at least one outgoing edge. The *level* of a node n in an AIG G, is its topological level from PIs. With established logic synthesis tools, any general boolean function can be transformed into an AIG representation, which helps to better understand the functioning of logic circuits.

A cut X of an AIG node n is defined as a set of nodes, called *leaves*, such that each path from a PI to n passes through at least one leaf. With node n and its cut X, we can get a LUT, and the node n is the *root* of the LUT. A cut is said to be K-feasible if the number of nodes in it does not exceed K. And the *subgraph* $S(X)$ *covered* by X is the induced subgraph containing nodes of paths from leaves to the root n. If n is an AND node with p and q as its two fanin nodes, its *K-feasible cut set* $\Phi(n)$ can be enumerated by $\Phi(p)$ and $\Phi(q)$:

$$\Phi(n) = \{\{n\}\} \cup \{X \cup Z | X \in \Phi(p), Z \in \Phi(q), |X \cup Z| \leq K\}$$

Using this cut-enumeration technique in the topological order, K-feasible cuts for all nodes can be obtained. Starting from the POs and following an inverse topological order, we recursively convert K-feasible cuts into K-input LUTs until reach the PIs, we can finally obtain a vaild LUT-mapping.

State-of-the-art Method – IfMap. Implemented in ABC, IfMap has two mapping modes: level-priority and area-priority. The former first achieves the optimal level and then optimizes the area with the level constraint, while the latter focuses directly on area optimization without considering the level.

Applying the priority cuts method, for an AND node n with fanins p and q, its *cut set*, denoted as $\phi(n)$, stores C K-feasible cuts, and its *best cut*, denoted as $X^*(n)$, is used to be converted into a LUT. We use $\phi(p)$ and $\phi(q)$ to enumerate a subset of $\Phi(n)$, denoted as $\Phi_s(n)$. In each iteration, a *sorting criteria* is chosen, under which the top C cuts among $\Phi_s(n)$ are stored in $\phi(n)$ and the very top one is a candidate for the best cut.

There are some notations to represent the LUT-mapping result. $LUT(n)$ denotes the LUT whose root is n (empty for those who are not roots of any mapped LUTs). The *reference number*, denoted as $R(n)$, is defined as the fanout number of $LUT(n)$ (zero if $LUT(n)$ is empty). We *remove* $LUT(n)$ by subtracting one from the reference numbers of n's leaves, and remove those leaves' LUTs with zero fanout recursively. We *construct* $LUT(n)$ by doing the opposite.

The area utility value, an indicator of the area property, can be calculated either from a global view or from a local view.

*1) **Global view (Global Area Flow)**:* For a cut X, its *global utility* $U_g(X)$ is defined as follows

$$U_g(X) = \sum_{n \in X} \frac{U_g(n)}{R(n)^1} + 1. \tag{1}$$

Here $U_g(n) = 0$ for PIs and $U_g(n) = U_g(X^*(n))$ for others.

Before the first iteration, the reference number $R(n)$ is initialized to the fanout of n, and after each iteration, it is updated according to the LUT-mapping result. By calculating the utility $U_g(X)$ for all cuts following the topological order, the cut sets as well as the best cuts of all nodes are determined and finally the LUT-mapping are implemented.

*2) **Local view (Exact Local Area)**:* For a cut X, its *local utility* $U_l(X)$ is defined as follows

$$U_l(X) = \sum_{n \in X} U_l(n) * I(n) + 1. \tag{2}$$

Here $U_l(n) = 0$ for PIs and $U_l(n) = U_l(X^*(n))$ for others. $I(n) = 1$ when $R(n) = 0$ and $I(n) = 0$ otherwise.

[1]In practice, the $R(n)$ here is an estimate of the final reference number.

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

III. The Proposed EasyMap Method

In this section, we first present a theoretical analysis of IfMap's fast convergence to suboptimal solutions, and then propose two new heuristic algorithms. We introduce new utility functions for both the global and local views, which capture the key features of cuts. By combining all heuristic algorithms and utility functions, we obtain 16 distinct heuristic approaches. To further optimize these approaches, we use black-box optimization techniques to determine their hyper-parameters and schedule them efficiently.

A. Analysis for traditional methods

In the Global Area Flow, if fanin p of a node n has an out-degree larger than one, it has high possibility to be selected as a part of the best cut of the root n. Moreover, such phenomenon persists over time, even in the Exact Local Area algorithm. To reveal such kind phenomenon, we first present the following proposition which indicates that under certain conditions some cuts are always preferred than others.

Proposition 1. *For two fanins p and q of a node n, given any cut pair $X_p \in \Phi(p)$ and $X_q \in \Phi(q)$ such that $p \notin X_q$, if*

$$[R(p)-1] * \sum_{v \in X_p \setminus X_q} \frac{U_g(v)}{R(v)} > \sum_{v \in X_p \cap X_q} \frac{U_g(v)}{R(v)} + 1,$$

then the global utility $U_g(X_1) < U_g(X_2)$ where $X_1 = \{p\} \cup X_q$ and $X_2 = X_p \cup X_q$.

Proof. From the definition of U_g:

$$U_g(X_1) = 1 + \frac{U_g(p)}{R(p)} + \sum_{v \in X_q} \frac{U_g(v)}{R(v)} \tag{3}$$

$$U_g(X_2) = 1 + \sum_{v \in X_p \setminus X_q} \frac{U_g(v)}{R(v)} + \sum_{v \in X_q} \frac{U_g(v)}{R(v)} \tag{4}$$

Since $U_g(p)$ is given by its best cut, we have:

$$U_g(p) \leq U_g(X_p) = 1 + \sum_{v \in X_p} \frac{U_g(v)}{R(v)} \tag{5}$$

Applying Eq. 5 in Eq. 4, we get:

$$U_g(X_1) \leq 1 + \frac{1 + \sum_{v \in X_p} \frac{U_g(v)}{R(v)}}{R(p)} + \sum_{v \in X_q} \frac{U_g(v)}{R(v)} \tag{6}$$

Joining Eq. 4 and Eq. 6, if

$$\sum_{v \in X_p \setminus X_q} \frac{U_g(v)}{R(v)} > \frac{1 + \sum_{v \in X_p} \frac{U_g(v)}{R(v)}}{R(p)} \tag{7}$$

which is equivalent to:

$$[R(p)-1] * \sum_{v \in X_p \setminus X_q} \frac{U_g(v)}{R(v)} > \sum_{v \in X_p \cap X_q} \frac{U_g(v)}{R(v)} + 1 \tag{8}$$

we can deduce $U_g(X_1) < U_g(X_2)$. □

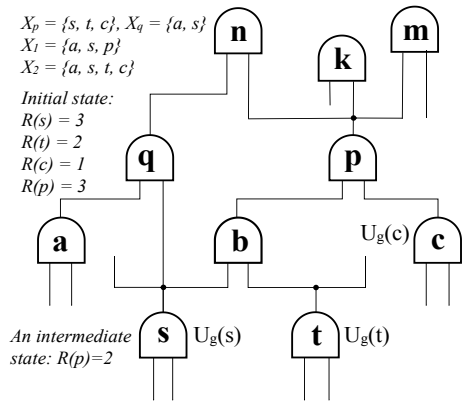

Fig. 2. An illustration of an AIG.

The condition of Proposition 1 is easy to meet. The reference number is initialized to the fanout number in the AIG. In many situations, X_p and X_q share *no nodes or only few nodes*, which means $|X_p \setminus X_q|$ is much bigger than $|X_p \cap X_q|$. Besides, the U_g of nodes can easily reach dozens, hundreds or even more as level increases. Hence if the fanout number of p is greater than one, it is of great possibility that the condition of Proposition 1 is satisfied and thus $U_g(X_1) < U_g(X_2)$, leading p to appear in the best cut of n in the global area flow.

For instance, let's consider an AIG with the root node n having two fanins p and q, as shown in Fig. 2. Let $X_q = \{a, s\}$ and $X_p = \{s, t, c\}$. Initially, $R(s) = 3$, $R(t) = 2$, $R(c) = 1$, $R(p) = 3$. The condition of the Proposition 1 can be simplified to $U_g(s) < 3U_g(t) + 6U_g(c) - 3$. This condition can be met easily if $U_g(s)$, $U_g(t)$, and $U_g(c)$ are of the same order of magnitude. Therefore, $X_1 = \{a, s, p\}$ is preferred over $X_2 = \{a, s, t, c\}$.

However, if we select X_2 instead of X_1, not only can the LUT cover more nodes, but the LUT can also share leaves s and t with many other LUTs, which shows a potentially better choice to reduce the area and restrict the number of edges.

As previously discussed, in situations described by Proposition 1, the global area flow tends to select cuts with a fixed pattern, ignoring other potential solutions. In addition, for the Exact Local Area, when $R(p) > 0$, the node p does not contribute to the utility of cuts, making it free to be placed into any best cut. This also leads to situations where X_1 is preferred over X_2 in the Exact Local Area, making it difficult to optimize in these cases. Therefore, both the Global Area Flow and Exact Local Area can get stuck in these situations, leading to fast convergence of the mapping algorithm.

B. New heuristic algorithms

To tackle the issue discussed in section III-A, we propose two new algorithms: Cut Filter in Global view and Causality Mask in Local view.

*1) **Cut Filter in Global view (CFG):*** First, for a cut X, we introduce three variables $nShare(X)$, $tDist(X)$ and $aLev(X)$, that are the total number of fanouts of X's leaves, the total distance between its root and any of the cut leaves

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Cut Filter in Global view

Input: node n (excluding PIs) and its best cut $X_0^*(n)$, cut sets of its two fanins: $\phi(p)$ and $\phi(q)$, threshold θ
Output: updated cut set and best cut of n: $\phi(n)$ and $X^*(n)$

```
Set φ(n) and φ'(n) as empty;
Enumerate Φ_s(n) by φ(p) and φ(q);
for cut X in Φ_s(n) ∪ {X_0^*(n)} do
    Calculate U_g(X);
    if Score(X, n) < θ then
        insert X into φ(n) ordered by GSort;
    else
        insert X into φ'(n) ordered by GSort;
    end
end
while φ(n) is not full and φ'(n) is not empty do
    pop the first cut X' in φ'(n) and insert X' into φ(n)
      ordered by GSort;
end
The first cut in φ(n) is selected as X*(n) (when with the
  level constraint, select X_0^*(n) if the first cut increases the
  overall level);
return φ(n) and X*(n)
```

and the average level of the cut leaves respectively. We then define a score function for a cut X as follows:

$$Score(X) = aLev(X)^{\gamma_1} * nShare(X)^{\gamma_2} * tDist(X)^{\gamma_3} \quad (9)$$

where $\gamma_1 > 0$, $\gamma_2 < 0$ and $\gamma_3 < 0$ are three constants.

We filter cuts using the score function and discard cuts with large scores. See Alg. 1 for details, where U_g lies in the first position of $GSort$. Cuts with large $nShare$, $tDist$ and small $aLev$ get a small score and thus are more likely to be selected.

Now lets consider the situation of Proposition 1. Note that nodes in $X_p \setminus X_q$ contribute larger $tDist$ and smaller $aLev$ than p does. And usually $X_p \setminus X_q$ contains more than one node, so $nShare(X_p \setminus X_q) > nShare(\{p\})$ happens with high possibility. Besides, the higher the level of the root n is, the larger the $aLev$ will be, and thus the larger the difference between $Score(X_2)$ and $Score(X_1)$ will be. It is evident that in the global score filter algorithm, X_2 is more likely to be discarded than X_1, especially when n is in a high level, which can help mapping algorithms to escape from local optima.

*2) **Causality Mask in Local view (CML):*** Note that Exact Local Area gets stuck in the situation described in Proposition 1, where a node's utility becomes zero when its reference number is larger than zero. This means that if a node is shared by multiple LUTs, it will be considered cost-free for any of its roots, and hence it contributes nothing to the final result's area and is very likely to be chosen as a root again.

To alleviate this limitation easily leading to local optima, we propose the Causality Mask in Local view, which resets all reference numbers to zero, instead of using the ones after the last iteration, at the beginning of each iteration and updates them dynamically. Here the notations with the subscript '0' represent those got from the last iteration. In each iteration, we traverse nodes n in the topological order to determine their cut sets and best cuts, and finally implement the LUT-mapping from POs using best cuts, including updating $R(*)$ and $LUT(*)$. Alg. 2 shows the procedure to determine cut sets and best cuts, where U_l lies in the first position of $LSort$.

For a node v, let $Roots(v) = \{u \in V(G)|$ v is a leaf of $LUT_0(u)\}$.We say a node n *dominates* v, if $n \in Roots(v)$ and has the least topological order. In addition, we say n is the *dominating root* of v and v is a *dominated leaf* of n.

We fix $R_0(*)$ and change $R(*)$. $R(*)$ instead of $R_0(*)$ are used to calculate the local utility values U_l. When $R_0(n) > 0$, two more things will be done: before the determination, we subtract one from the $R(*)$ of n's dominated leaves and remove the LUTs whose root reference numbers become zero; after the determination, we construct $LUT(n)$ and add one to $R(n)$. Note that the added $R(n)$ will be subtracted by n's dominating root. The addition and subtraction trick masks the influences from the roots of n other than its dominating root, reducing the phenomenon that n is cost-free for all roots and thus helping the algorithm escape the local optima.

For the example in Fig. 2, suppose we have finished the global view heuristic and get an intermediate state, and assume that in the result of previous round of LUT mapping, m and n are the roots of p. Among them, n is the dominating root of p. In the current round, it is possible for k , whose topological order precedes that of point n, to become a root of p. In traditional algorithms, the value of $R(p)$ for this round would be set to 2. Therefore, p appears cost-free to points k, m and n, and its area would not impact the final result. However, in our proposed CML, we copy R to R_0 and initialize $R(p) = 0$ for this round. Upon traversing to p, as $R_0(p)$ from the previous round is greater than 0, $R(p)$ is incremented by one, resulting in $R(p) = 1$ at this point. If, in this round, k does not select p as a cut-leaf, when computing the utility function for n in this round, $R(p)$ will be subtracted by 1, effectively making $R(n) = 0$. Consequently, p is no longer cost-free for n. Differently, if k selects p as a cut-leaf in this round, due to our dynamic update this round, $R(p)$ is incremented by one, being 2, when calculating for k. So when computing the utility function for n, $R(p) = 1$ after the substraction. As a result, p is still cost-free for n in this scenario. From the given example, we can see that p's "cost-free for all" property can be altered if no nodes preceding the dominating root n select p as a cut-leaf during this round. Since no preceding nodes selecting p shows a negative area attribute associated with p, the alteration is critical in improving the area.

C. Utility function with new costs

It can be observed from Eq. 1 and Eq. 2 that the utility functions of a cut X are contributed by two components: a sum regarding leaves' utility and a constant 1. The constant 1 is a cost function measures the contribution of the cut itself, when it is mapped into a LUT, to the final area. Although the second component 1 is a reasonable cost function, but it is not powerful enough to represent the cut itself. We *decompose* the constant 1 into three ingredients that capture different characteristics of the cut itself , which provides us with optimization strategies in three gradients directions.

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Causality Mask in Local view

Input: an AIG G, original reference numbers $R_0(*)$ and best cuts $X_0^*(*)$ for nodes in G
Output: updated cut sets $\phi(*)$ and best cuts $X^*(*)$ for nodes in G

```
Set φ(*), R(*), LUT(*) as empty;
for node n of G in topological order do
    if n is a Primary Input then
        continue;
    end
    if R_0(n) > 0 then
        for node v in X_0^*(n) do
            if v is a Primary Input then
                continue;
            end
            if n dominates v then
                R(v) = R(v) − 1;
                if R(v) == 0 then
                    remove LUT(v);
                end
            end
        end
    end
    Enumerate Φ_s(n) by φ(p) and φ(q) (two fanins);
    for cut X in Φ_s(n) ∪ {X_0^*(n)} do
        Calculate U_l(X);
        insert X into φ(n) ordered by LSort;
    end
    Select the first cut in φ(n) as X^*(n) (when with the
      level constraint, select X_0^*(n) if the first cut increases
      the overall level);
    if R_0(n) > 0 then
        construct LUT(n) corresponding to X^*(n);
        R(n) = R(n) + 1;
    end
end
return φ(*) and X^*(*)
```

We consider three characteristics of a cut X: the *volume* $Vol(X)$: number of nodes covered by X (including the root while not including the cut leaves), $nLeaves(X)$: number of cut leaves, and the *redundancy* $Redun(X)$ that is the number of redundancy edges (extra edges added to a spanning tree) of the subgraph $S(X)$ covered by X. The combination of these three characteristics together forms the constant 1.

Proposition 2. *nLeaves(X) - Vol(X) + Redun(X) = 1.*

Proof. Let $|E|$ be the total number of edges in the subgraph $S(X)$ covered by the cut X. Since the in-degree of each node is 2, we have $|E| = 2 * Vol(X)$. And by the definition of redundancy, we have $Redun(X) = |E| - [Vol(X) + nLeaves(X) - 1]$. Combining the above two equations, we can obtain this proposition. □

By focusing on different characteristics, we deduct three different cost functions of a cut X.

1) ***Vol:*** Greedily, we hope that each cut covers as many nodes as possible, which intuitively will reduce the number of LUT needed to cover the whole graph. Following this, we define a cost function:

$$C_v(X) = \frac{1}{Vol(X) + \alpha}. \tag{10}$$

2) ***nLeaves:*** A LUT with more inputs can express more powerful logical functionality. So if cuts with larger $nLeaves$ are selected, the number of LUTs needed to cover the logic network will possibly be reduced. However, a cut with more leaves number consumes more edges, so if we keep giving a higher priority to cut with a larger nLeaves, the edge will grow explosively.

To reduce the area and restrict the number of edges meanwhile, we propose the following cost function:

$$C_n(X) = c_k, \tag{11}$$

where c_k is a constant and k is the $nLeaves$ of X.

If we set the value of c_k to monotonically decrease/increase with an increase in k, we will tend to prefer cuts with a larger/smaller number of nLeaves. We can apply decreased c_k to achieve more area reduction, and then choose increased c_k in the next round to control the growth of the edges. Fig. 3 shows the area curve and the edge curve respectively. If mapper performs Global Area Flow in the first round and Exact Local Area in the following two rounds. With easy mapper, we remain the first global view round and adopt decreased c_k and increased c_k sequentially in the following two local view rounds.

3) ***Redundancy:*** The abundance of the extra edges shows a higher information density of a LUT. And from Proposition 2, we can see that large redundancy of a cut X leads to small nLeaves and large volume, in other words, its corresponding LUT can cover more nodes with less cut leaves. Based on such character of redundancy, we define a new cost function:

$$C_r(X) = \frac{1}{Redun(X) + \beta}. \tag{12}$$

For convenience, we denote the original constant 1 in utility functions in Global Area Flow and Exact Local Area as cost function C_o.

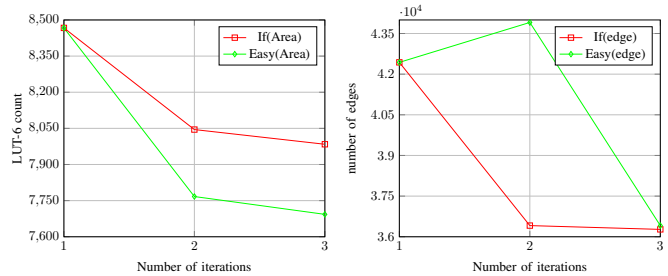

Fig. 3. The relationship between iteration rounds and the area/edge.

D. A pool of heuristic approaches

We can generate a heuristics pool $\mathcal{P}$ by combining all four algorithms (Global Area Flow (GAF), Exact Local Area (ELA), Cut Filter in Global view (CFG), and Causality Mask in Local view (CML)) with all four cost functions (C_v, C_n, C_r, and C_o), resulting in a total of 16 heuristic approaches.

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

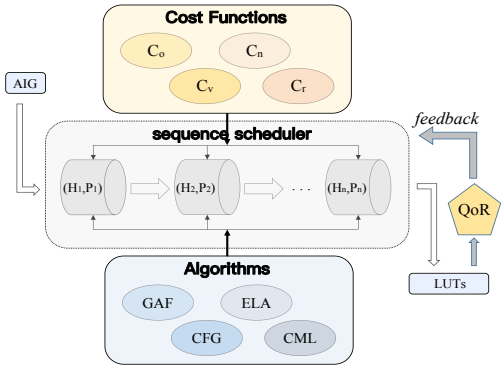

Fig. 4. The algorithm flow of the proposed EasyMap.

Table I
An example of optimization sequence.

algorithm + cost function	hyper-parameters
$GAF + C_v$	α
$CML + C_o$	$none$
$CFG + C_r$	$\gamma_1, \gamma_2, \gamma_3, \beta$
$ELA + C_n$	$c_2, ..., c_k$

E. Sequence Scheduling

With the heuristic approach pool $\mathcal{P}$, it is possible to schedule these approaches and obtain different optimization sequences. By using black-box optimization techniques, we can conduct adaptive sequence scheduling and determine the hyper-parameters of different heuristic approaches.

Let $\{H_1, H_2, \ldots, H_n\}$ be a sequence of heuristic approaches, where each H_i consists of an algorithm, a cost function, and its corresponding hyper-parameters P_i. For instance, if a heuristic sequence is $\{GAF + C_v, CML + C_o, CFG + C_r, ELA + C_n\}$ in Table I, we set the hyper-parameters and then apply Global Area Flow with cost C_v, Causality Mask in Local view with cost C_o, Cut Filter in Global view with cost C_r, and Exact Local Area with cost function C_n sequentially. Finally, we denote an optimization sequence as

$$S = \{(H_1, P_1), (H_2, P_2), \ldots, (H_n, P_n)\}.$$

Finding a good synthesis result is a sequential decision problem. We take the synthesis process as a black-box, and the sequential decision problem as a black-box optimization problem. We employ Batch Sequential Black-box Optimization (BSBO) [21] as our black-box optimization technique, which is a Bayesian Optimization algorithm designed for logic synthesis [22], [23] and can determine algorithm sequences and their hyper-parameters at the same time, the process is shown as Fig. 4. We apply BSBO as follows:

1) BSBO recommends a branch of new sequences, then evaluated by the synthesis tool according to their QoR. If a sequence with a better evaluation is found, the new sequence is accepted and set as the current best sequence.
2) All known sequences and their evaluation results are used as training data to update the algorithm of BSBO.
3) It repeats iteratively until convergence or a sequence is found satisfying the pre-defined stopping criteria.

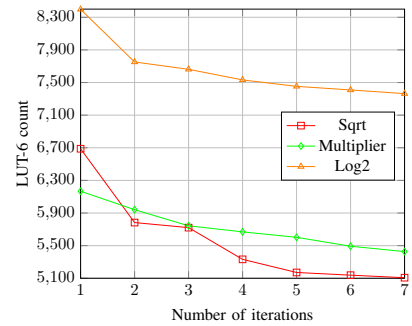

Fig. 5. The convergence curve of EasyMap on Sqrt, Multiplier, Log2 circuits of EPFL benchmark. We slow down the convergence with our approach to achieve better performance. Moreover, we can see EasyMap can always outperform IfMap after the same number (from 1 to 7) of iteration rounds.

In addition, considering its applicability, we have also provided a set of general heuristic sequences and their corresponding hyperparameters based on our experience, which saves the time and effort required for individually fine-tuning each circuit, thereby reducing the associated time overhead.

IV. EXPERIMENTS

We implement EasyMap using the logic synthesis tool ABC and test it on the EPFL benchmark with 20 circuits. These circuits are divided into two groups based on their functionality: arithmetic and random control. For hyper-parameters of the CFG in Alg. 1, we choose γ_1, γ_2, γ_3 from $[0, 0.5]$, $[-2.5, 0]$, $[-2.5, 0]$ respectively. For hyper-parameters of cost functions, we choose c_k, α, β from $[0.5, 2]$, $[5, 15]$, $[5, 15]$ respectively.

We compare IfMap and our EasyMap on various tasks, taking into account the following aspects:

- **Objectives:** We consider two objectives, namely area-oriented with level constraints and area-oriented without any constraints, and optimize for both cases.
- **Synthesis processes:** We compare, by replacing IfMap with EasyMap, in two different synthesis processes: using only {if} and using a combination of {lo, if, pm} where "lo" represents a set of logic optimization operators, e.g., rewrite; and "pm" represents a set of post-mapping operators, e.g., lutpack. The former conducts only mapping, while the latter conducts logic optimization, mapping, and post-mapping optimization.
- **Runtime:** We consider different runtime constraints and provide two types of EasyMap optimization sequences. The first will perform three iterations of heuristic algorithms to ensure that the average runtime difference compared to the default IfMap does not exceed 10%. The second performs heuristic algorithms until convergence.
- **Customization:** We use Bayesian optimization to obtain a customized sequence for each circuit. Considering some scenarios don't allow for the time cost of sequence and parameter optimization for each circuit, we also provide generic sequences applicable to all circuits for 3 iterations and convergence respectively. The sequence for 3 iterations is $\{GAF + C_0, CML + C_0,\ ELA + C_n$(increased)$\}$ with level constraint and $\{GAF{+}C_0, CML{+}C_0,\ CML{+}C_0\}$ without constraint.

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

Table II

Results of **optimizing the area with a level constraint**, comparing in the synthesis process **{if}**. "Customized" denotes that each circuit has a customized optimization sequence, while, while "Generic" denotes that all circuits use one generic optimization sequence. "3 iters" indicates that the optimization sequence has a length of 3, and "convergence" means that the optimization sequence runs until convergence.

{if}	**If**			**Customized EasyMap(3 iters)**				**Customized EasyMap(convergence)**				**Generic EasyMap(3 iters)**				**Generic EasyMap(convergence)**			
Benchmark	Lev	Area	Edge	Area	Impr.	Edge	Impr.	Area	Impr.	Edge	Impr.	Area	Impr.	Edge	Impr.	Area	Impr.	Edge	Impr.
Adder	51	254	1036	254	0%	1044	-0.8%	254	0%	1020	1.5%	254	0%	1068	-3.1%	254	0%	1036	0%
Bar	4	512	2688	512	0%	2688	0%	512	0%	2688	0%	512	0%	2688	0%	512	0%	2688	0%
Div	864	22031	77681	18500	16.0%	81952	-5.5%	18028	18.2%	81991	-5.5%	19000	13.8%	81529	-4.9%	18033	18.1%	83267	-7.2%
Hyp	4194	44503	198037	40436	9.1%	201952	-2.0%	40383	9.3%	201853	-1.9%	40565	8.8%	202338	-2.2%	40639	8.7%	201277	-1.6%
Log2	77	7973	36309	7468	6.3%	35793	1.4%	7219	9.5%	34917	3.8%	7511	5.8%	35859	1.2%	7341	7.9%	35737	1.6%
Max	56	842	3350	812	3.6%	3308	1.3%	802	4.8%	3311	1.2%	831	1.3%	3597	-7.3%	826	1.9%	3607	-7.7%
Multiplier	53	5913	27696	5560	6.0%	27739	-0.2%	5299	10.4%	27888	-0.7%	5627	4.8%	27874	-0.6%	5452	7.8%	29349	-6.0%
Sin	42	1458	6486	1389	4.7%	6443	0.7%	1303	10.6%	6740	-3.9%	1388	4.8%	6652	-2.6%	1318	9.6%	6706	-3.4%
Sqrt	1033	5720	26931	5137	10.2%	25880	3.9%	5075	11.3%	25786	4.3%	5163	9.7%	25964	3.6%	5078	11.2%	25873	3.9%
Square	50	3985	16501	3470	12.9%	17230	-4.2%	3327	16.5%	17199	-4.2%	3523	11.6%	17247	-4.5%	3397	14.8%	17040	-3.2%
Average Impr.					**6.9%**		**-0.5%**		**9.1%**		**-0.5%**		**6.1%**		**-2.0%**		**8%**		**-2.4%**
Arbiter	18	2722	15137	2722	0%	15137	0%	2722	0%	15137	0%	2722	0%	15137	0%	2722	0%	15137	0%
Cavlc	4	122	628	122	0%	622	1%	120	1.6%	624	0.6%	122	0%	627	0.2%	121	0.8%	639	-1.8%
Ctrl	2	29	136	29	0%	136	0%	29	0%	136	0%	29	0%	136	0%	29	0%	138	-1.5%
Dec	2	287	684	287	0%	684	0%	287	0%	684	0%	287	0%	684	0%	287	0%	684	0%
Int2float	3	49	256	49	0%	256	0%	49	0%	256	0%	49	0%	256	0%	49	0%	257	-0.4%
I2c	4	365	1668	359	1.6%	1683	-0.9%	356	2.5%	1705	-2.2%	362	0.8%	1704	-2.2%	362	0.8%	1749	-4.9%
Mem_ctrl	25	12096	54487	11868	1.9%	54629	-0.3%	11624	3.9%	55316	-1.5%	11938	1.3%	55294	-1.5%	11732	3.0%	55719	-2.3%
Priority	31	219	1074	215	1.8%	1078	-0.4%	211	3.7%	1117	-4.0%	217	0.9%	1079	-0.5%	224	-2.3%	1071	0.3%
Router	11	91	306	90	1.1%	309	-1.0%	89	2.2%	307	-0.3%	91	0%	306	0%	91	0%	312	-2.0%
Voter	17	2818	13598	2197	22.0%	12449	8.4%	2116	24.9%	11477	15.6%	2629	6.7%	13788	-1.4%	2582	8.4%	13776	-1.3%
Average Impr.					**2.8%**		**0.7%**		**3.9%**		**0.8%**		**1.0%**		**-0.5%**		**1.1%**		**-1.4%**

Table III

The results of experiments **optimize the area with a level constraint**, comparing in the process **{lo, if, pm}**.

{lo, if, pm}	**If**			**Customized EasyMap(3 iters)**				**Customized EasyMap(convergence)**				**Generic EasyMap(3 iters)**				**Generic EasyMap(convergence)**			
Benchmark	Lev	Area	Edge	Area	Impr.	Edge	Impr.	Area	Impr.	Edge	Impr.	Area	Impr.	Edge	Impr.	Area	Impr.	Edge	Impr.
Adder	51	254	1036	246	3.1%	1040	-0.4%	245	3.5%	1019	1.6%	245	3.5%	1050	-1.4%	245	3.5%	1068	-3.1%
Bar	4	512	2688	512	0%	2688	0%	512	0%	2688	0%	512	0%	2688	0%	512	0%	2688	0%
Div	864	9989	40694	9287	7.0%	41134	-1.1%	8833	11.6%	41597	-2.2%	9512	4.8%	42419	-4.2%	8934	10.6%	41446	-1.8%
Hyp	4217	44876	198354	40570	9.6%	202649	-2.2%	40395	10.0%	202114	-1.9%	40591	9.5%	202704	-2.2%	40796	9.1%	201682	-1.7%
Log2	68	8164	37363	7777	4.7%	36512	2.3%	7353	9.9%	36077	3.4%	7826	4.1%	37082	0.8%	7462	8.6%	36396	2.6%
Max	44	855	3201	832	2.7%	3287	-2.7%	821	3.5%	3376	-4.7%	828	3.2%	3321	-3.7%	833	2.6%	3582	-11.9%
Multiplier	53	5872	27050	5449	7.2%	27627	-2.1%	5297	9.8%	27488	-1.6%	5566	5.2%	27270	-0.8%	5417	7.7%	27182	-0.5%
Sin	36	1502	6867	1450	3.5%	6844	0.3%	1392	7.3%	6803	0.9%	1450	3.5%	6858	0.1%	1400	6.8%	6850	0.2%
Sqrt	1032	3419	16456	3385	1.0%	16480	-0.1%	3332	2.5%	16377	0.5%	3408	0.3%	16539	-0.5%	3341	2.3%	16439	0.1%
Square	50	3924	16344	3423	12.9%	16978	-4.2%	3334	15.0%	16940	-3.6%	3462	11.8%	17153	-4.9%	3382	13.8%	16881	-3.3%
Average Impr.					**5.2%**		**-1.0%**		**7.3%**		**-0.8%**		**4.6%**		**-1.7%**		**6.5%**		**-1.9%**
Arbiter	18	2718	15133	2718	0%	15133	0%	2718	0%	15133	0%	2718	0%	15133	0%	2719	0%	15134	0%
Cavlc	4	131	677	129	1.5%	677	0%	127	3.1%	672	0.7%	131	0%	676	0.1%	130	0.8%	690	-1.9%
Ctrl	2	29	134	29	0%	134	0%	29	0%	134	0%	30	-3.0%	133	0.7%	29	0%	134	0%
Dec	2	287	684	287	0%	684	0%	287	0%	684	0%	287	0%	684	0%	287	0%	684	0%
Int2float	3	49	234	49	0%	234	0%	49	0%	234	0%	49	0%	234	0%	49	0%	235	-0.4%
I2c	4	327	1428	317	3.1%	1420	0.6%	309	5.5%	1367	4.3%	323	1.0%	1412	1.1%	317	3.1%	1400	2%
Mem_ctrl	21	10623	46254	10542	0.8%	46106	0.3%	10230	3.7%	46739	-1.0%	10480	1.0%	46699	-1.0%	10384	2.2%	47308	-2.3%
Priority	28	186	868	169	9.1%	786	9.4%	164	11.8%	782	9.9%	180	3.0%	857	1.3%	178	4.3%	831	4.3%
Router	6	74	210	70	5.4%	194	7.6%	69	6.8%	192	8.6%	74	0%	211	-0.5%	71	4.1%	201	4.3%
Voter	13	1644	7817	1495	9.1%	7709	1.4%	1425	13.3%	7409	5.2%	1541	6.0%	7792	0.3%	1490	9.4%	7733	1.1%
Average Impr.					**2.9%**		**1.7%**		**4.4%**		**2.8%**		**0.8%**		**0.2%**		**2.4%**		**0.7%**

Table IV

The results of experiments **optimize the area without level constraints**, comparing in the synthesis process **{if -a}**.

{if}	**If**		**Customized EasyMap(3 iters)**				**Customized EasyMap(convergence)**				**Generic EasyMap(3 iters)**				**Generic EasyMap(convergence)**			
Benchmark	Lev	Area	Lev	Impr.	Area	Impr.	Lev	Impr.	Area	Impr.	Lev	Impr.	Area	Impr.	Lev	Impr.	Area	Impr.
Adder	121	249	64	47.1%	192	22.5%	64	47.1%	192	22.5%	64	47.1%	192	22.9%	64	47.1%	192	22.9%
Bar	4	512	4	0%	512	0%	4	0%	512	0%	4	0%	512	0%	4	0%	512	0%
Div	2117	23863	1223	42.2%	15378	35.6%	1161	45.2%	15243	36.1%	1218	42.6%	15364	35.6%	1129	46.7%	15383	35.5%
Hyp	8401	47558	5064	39.7%	40050	15.8%	4360	48.1%	**39781**	16.4%	5510	34.4%	40301	15.3%	4576	45.5%	39950	16.0%
Log2	159	7581	117	26.4%	6811	10.0%	133	16.4%	**6530**	13.9%	132	17.0%	6904	9.0%	125	21.4%	6816	10.1%
Max	115	721	108	6.1%	708	1.8%	93	19.1%	721	0%	111	3.5%	711	1.4%	83	27.8%	755	-4.5%
Multiplier	126	5678	122	3.2%	4876	14.1%	93	26.2%	**4432**	21.9%	89	29.4%	5070	10.7%	98	22.2%	4838	14.8%
Sin	74	1444	58	21.6%	1270	12.0%	52	29.7%	**1216**	15.8%	59	20.3%	1297	10.2%	55	25.7%	1253	13.2%
Sqrt	3954	8084	1169	70.4%	4494	44.4%	1096	72.3%	4461	44.8%	1138	71.2%	4498	44.4%	1119	71.7%	4472	44.7%
Square	122	3994	81	33.6%	3363	15.8%	67	45.1%	**3263**	18.3%	81	33.6%	3416	14.5%	70	42.6%	3305	17.3%
Average Impr.				**29.0%**		**17.2%**		**34.9%**		**19.0%**		**29.9%**		**16.4%**		**35.1%**		**17.0%**
Arbiter	19	2599	19	0%	2594	0%	19	0%	2594	0.2%	19	0%	2594	0.2%	19	0%	2719	-4.6%
Cavlc	6	116	5	16.7%	116	0%	5	16.7%	116	0%	6	0%	116	0%	5	16.7%	120	-3.4%
Ctrl	2	29	2	0%	29	0%	2	0%	29	0%	2	0%	29	0%	2	0%	29	0%
Dec	2	287	2	0%	272	5.2%	2	0%	272	5.2%	2	0%	272	5.2%	2	0%	272	5.2%
Int2float	5	47	4	0%	47	0%	4	20.0%	47	0%	5	0%	47	0%	4	20.0%	47	0%
I2c	7	353	5	28.6%	348	1.4%	5	28.6%	347	1.7%	6	14.3%	351	0.6%	5	28.6%	351	0.6%
Mem_ctrl	53	11631	38	28.3%	11444	1.6%	34	35.8%	11360	2.3%	42	20.8%	11536	0.8%	38	28.3%	11345	2.5%
Priority	101	264	32	68.3%	216	18.2%	31	69.3%	215	18.6%	32	68.3%	216	18.2%	31	69.3%	226	14.4%
Router	17	73	14	17.6%	71	2.7%	13	23.5%	70	4.1%	14	17.6%	73	0%	13	23.5%	73	0%
Voter	37	2744	24	35.1%	1945	29.1%	19	48.6%	1862	32.1%	25	32.4%	2140	22.0%	22	40.5%	1963	28.5%
Average Impr.				**19.5%**		**5.8%**		**24.3%**		**6.4%**		**15.3%**		**4.7%**		**22.7%**		**4.3%**

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

Table V
The results of experiments **optimize the area without level constraints**, comparing in the process **{lo, if -a, pm}**.

{lo, if, pm}	**If**		**Customized EasyMap(3 iters)**				**Customized EasyMap(convergence)**				**Generic EasyMap(3 iters)**				**Generic EasyMap(convergence)**			
Benchmark	Lev	Area	Lev	Impr.	Area	Impr.	Lev	Impr.	Area	Impr.	Lev	Impr.	Area	Impr.	Lev	Impr.	Area	Impr.
Adder	121	249	64	47.1%	192	22.9%	64	47.1%	192	22.9%	64	47.1%	192	22.9%	64	47.1%	192	22.9%
Bar	4	512	4	0%	512	0%	4	0%	512	0%	4	0%	512	0%	4	0%	512	0%
Div	2082	8176	1146	45.0%	6414	21.6%	1108	46.8%	6291	23.1%	1209	41.9%	6584	19.5%	1128	45.8%	6340	22.5%
Hyp	8464	47968	4665	44.9%	40360	15.9%	4508	46.7%	39616	17.4%	5470	35.4%	40450	15.7%	4612	45.5%	40131	16.3%
Log2	154	7724	107	30.5%	7052	8.7%	124	19.5%	6452	16.5%	107	30.5%	7130	7.7%	102	33.8%	6889	10.8%
Max	89	742	81	9.0%	735	0.9%	75	15.7%	735	0.9%	96	-7.9%	741	0.1%	72	19.1%	751	-1.2%
Multiplier	127	5707	78	38.6%	5077	11.0%	103	18.9%	4380	23.3%	88	30.7%	5865	-2.8%	101	20.5%	4964	13.0%
Sin	69	1468	53	23.2%	1298	11.6%	52	24.6%	1216	15.8%	53	23.2%	1338	8.9%	48	30.4%	1270	13.5%
Sqrt	2000	4043	1142	42.9%	3282	18.8%	1053	47.4%	3124	22.7%	1358	32.1%	3435	15%	1076	46.2%	3161	21.8%
Square	122	3901	85	30.3%	3299	15.4%	65	46.7%	3232	17.1%	84	31.1%	3330	14.6%	64	47.5%	3264	16.3%
Average Impr.				**31.2%**		**12.7%**		**31.3%**		**16.0%**		**26.4%**		**10.2%**		**33.6%**		**13.6%**
Arbiter	19	2590	19	0%	2590	0%	19	0%	2590	0%	19	0%	2590	0%	19	0%	2591	0%
Cavlc	7	130	6	14.3%	126	3.1%	5	28.6%	125	3.8%	6	14.3%	128	1.5%	5	28.6%	128	1.5%
Ctrl	3	30	2	33.3%	29	3.3%	2	33.3%	29	3.3%	2	33.3%	29	3.3%	2	33.3%	29	3.3%
Dec	2	287	2	0%	272	5.2%	2	0%	272	5.2%	2	0%	272	5.2%	2	0%	272	5.2%
Int2float	5	41	5	0%	41	0%	5	0%	41	0%	5	0%	41	0%	4	20.0%	42	-2.4%
I2c	6	318	6	0%	310	2.5%	5	16.7%	305	4.1%	6	0%	320	-0.6%	5	16.7%	308	3.1%
Mem_ctrl	44	10519	33	25.0%	10101	4%	31	29.5%	10011	4.8%	37	15.9%	10176	3.3%	31	29.5%	10080	4.2%
Priority	87	222	30	65.5%	164	26.1%	29	66.7%	156	29.7%	30	65.5%	174	21.6%	30	65.5%	180	18.9%
Router	10	80	8	20.0%	70	12.5%	8	20.0%	68	15.0%	8	20.0%	76	5.0%	8	20.0%	78	2.5%
Voter	18	1820	17	5.6%	1362	25.2%	15	16.7%	1306	28.2%	17	5.6%	1362	25.2%	15	16.7%	1369	24.8%
Average Impr.				**16.4%**		**8.2%**		**21.2%**		**9.4%**		**15.5%**		**6.5%**		**23.0%**		**6.1%**

By combining these different experiment settings, we have 16 types of tasks to compare IfMap and EasyMap.

Before presenting details of our results, let's compare the convergence curve of EasyMap in Fig. 5 and IfMap in Fig. 1. EasyMap can not only optimize after few iterations contrast to IfMap, but also obtain better results than IfMap, which shows the effectiveness of our algorithm.

We use "genric" and "customized" to denote one common sequence for all circuits and customized sequence for each circuit; "3 iters" and "convergence" to denote a short sequence with 3 heuristic algorithms and a long sequence running until convergence. Table II and Table III present results optimizing area with the level constraint. Since IfMap and EasyMap achieve best level, and edge number is an important metric in the physical design, we omit the number of level and count edges here. We can see that the Generic EasyMap with 3 iterations improve upon IfMap by 6.1% and 4.6% in area with edge degradation around 2% regarding the arithmetic circuits in the two processes respectively, which shows the generic power of the exploration-enhanced heuristics. Compared to the Generic EasyMap with 3 iterations, the improvement of Generic EasyMap until convergence reaches 8% and 6.5% in area with only 0.5% edge degradation regarding the arithmetic circuits, indicating the deferred convergence and more optimization potential with the adaptive sequencing. Compared to the generic EasyMap (both "3 iters" and "convergence"), the customized EasyMap gets about 1% more area improvement regarding arithmetic circuits and reduce the edge degradation to no more than 1% by tapping more potential for each circuit and better balancing the area and the edge.

Table IV and Table V present the results of optimizing area without level constraint. We can see that the generic EasyMap with 3 iterations improve upon IfMap by 16.4%, 10.2% in area and 29.9%, 26.4% in level regarding arithmetic circuits in the two different process respectively. Further improvements can be obtained by running until convergence and by customization. In particular, with the customized EasyMap until convergence, we can improve upon IfMap by 19%, 16% in area and 34.9%, 31.3% in level regarding arithmetic circuits. It is worth mentioning that even by **running only the EasyMap operator** without introducing any other operators, we can **approach or even outperform the best results** of Hyp, Log2, Multiplier, Sin and Square circuits on the EPFL benchmark in terms of area metric in 2022 [24]. Besides, the average improvement regarding random control circuits can also reach 6.4%, 9.4% in area and 24.3%, 21.2% in level.

Upon reviewing the results in Table IV, we observe a close correlation between the improvement on the baseline and the circuit level and size. In Section III-A, we analyze how the original heuristic algorithm's selection of the best cut is often unreasonable in many cases. The larger the circuit size is, the more unreasonable decision is likely to happen. Since priority cuts at higher-level nodes are derived from lower-level nodes, any unreasonable selections propagate and accumulate as level goes high. Consequently, when circuits have smaller levels and sizes, the occurrence and the accumulation of unreasonable selections by IfMap is inherently little, and in some cases it even approaches the optimal mapping solution. Besides, the level constraint limits the update of many cuts with better area, so the room for area improvement in these circuits is further limited. Hence, the effectiveness of our method is not as pronounced in improving most circuits in random control circuits especially with the level constraint.

V. Conclusion

In this paper, we have proposed two new heuristic algorithms based on our proposed theoretical analysis of IfMap's convergence behavior. Furthermore, we design new utility functions that consider different features of cuts used to rank cuts. Together, we introduce a suite of heuristic approaches and use both our experimental experience and the black-box optimization techniques to schedule the heuristic algorithm sequence for improved performance. Our new mapping method, EasyMap, achieves significant improvements compared to IfMap. Actually, there is still some potential for improving EasyMap through more careful scheduling and parameter tuning with BSBO or other machine learning strategies.

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. Jang, B. Chan, K. Chung, and A. Mishchenko, "Wiremap: FPGA technology mapping for improved routability and enhanced LUT merging," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 2, no. 2, pp. 1–24, 2009.

[2] J. Cong and Y. Ding, "FlowMap: an optimal technology mapping algorithm for delay optimization in lookup-table based FPGA designs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 13, no. 1, pp. 1–12, 1994.

[3] D. Chen and J. Cong, "DAOmap: a depth-optimal area optimization mapping algorithm for FPGA designs," in *IEEE/ACM International Conference on Computer Aided Design, 2004. ICCAD-2004.*, Nov 2004, pp. 752–759.

[4] A. Mishchenko, S. Cho, S. Chatterjee, and R. Brayton, "Combinational and sequential mapping with priority cuts," in *2007 IEEE/ACM International Conference on Computer-Aided Design*, 2007, pp. 354–361.

[5] A. Mishchenko, S. Chatterjee, and R. K. Brayton, "Improvements to Technology Mapping for LUT-Based FPGAs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 2, pp. 240–253, 2007.

[6] L. Amarú, V. Possani, E. Testa, F. Marranghello, C. Casares, J. Luo, P. Vuillod, A. Mishchenko, and G. De Micheli, "LUT-Based Optimization For ASIC Design Flow," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 871–876.

[7] W. L. Neto, L. Amarú, V. Possani, P. Vuillod, J. Luo, A. Mishchenko, and P.-E. Gaillardon, "Improving LUT-based optimization for ASICs," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 421–426.

[8] D. B. West *et al.*, *Introduction to graph theory*. Prentice hall Upper Saddle River, 2001, vol. 2.

[9] V. Manohararajah, S. D. Brown, and Z. G. Vranesic, "Heuristics for area minimization in LUT-based FPGA technology mapping," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 11, pp. 2331–2340, 2006.

[10] J. Cong and Y.-Y. Hwang, "Simultaneous Depth and Area Minimization in LUT-based FPGA Mapping," in *Third International ACM Symposium on Field-Programmable Gate Arrays*, 1995, pp. 68–74.

[11] W. L. Neto, M. Austin, S. Temple, L. Amaru, X. Tang, and P.-E. Gaillardon, "LSOracle: A logic synthesis framework driven by artificial intelligence," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–6.

[12] W. Haaswijk, E. Collins, B. Seguin, M. Soeken, F. Kaplan, S. Süsstrunk, and G. De Micheli, "Deep Learning for Logic Optimization Algorithms," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2018, pp. 1–4.

[13] K. Zhu, M. Liu, H. Chen, Z. Zhao, and D. Z. Pan, "Exploring logic optimizations with reinforcement learning and graph convolutional network," in *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*, 2020, pp. 145–150.

[14] W. L. Neto, M. T. Moreira, Y. Li, L. Amarù, C. Yu, and P.-E. Gaillardon, "SLAP: A Supervised Learning Approach for Priority Cuts Technology Mapping," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 859–864.

[15] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Computer Aided Verification: 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings 22*. Springer, 2010, pp. 24–40.

[16] L. Amarú, P.-E. Gaillardon, and G. De Micheli, "The EPFL combinational benchmark suite," in *Proceedings of the 24th International Workshop on Logic & Synthesis (IWLS)*, no. CONF, 2015.

[17] X. Li, L. Chen, F. Yang, M. Yuan, H. Yan, and Y. Wan, "HIMap: a heuristic and iterative logic synthesis approach," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 415–420.

[18] C. Yang, Y. Xia, Z. Chu, and X. Zha, "Logic Synthesis Optimization Sequence Tuning Using RL-Based LSTM and Graph Isomorphism Network," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 8, pp. 3600–3604, 2022.

[19] G. Liu and Z. Zhang, "PIMap: A flexible framework for improving LUT-based technology mapping via parallelized iterative optimization," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 11, no. 4, pp. 1–23, 2019.

[20] A. Hosny, S. Hashemi, M. Shalan, and S. Reda, "DRiLLS: Deep Reinforcement Learning for Logic Synthesis," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2020, pp. 581–586.

[21] C. Feng, W. Lyu, Z. Chen, J. Ye, M. Yuan, and J. Hao, "Batch Sequential Black-box Optimization with Embedding Alignment Cells for Logic Synthesis," in *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2022, pp. 1–9.

[22] A. Grosnit, C. Malherbe, R. Tutunov, X. Wan, J. Wang, and H. B. Ammar, "BOiLS: Bayesian Optimisation for Logic Synthesis," in *2022 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2022, pp. 1193–1196.

[23] W. Lyu, F. Yang, C. Yan, D. Zhou, and X. Zeng, "Batch Bayesian optimization via multi-objective acquisition ensemble for automated analog circuit design," in *International conference on machine learning*. PMLR, 2018, pp. 3306–3314.

[24] "Best LUT-6 implementations-2022," 2022. [Online]. Available: https://github.com/lsils/benchmarks/tree/v2022.1/best_results

Authorized licensed use limited to: Peng Cheng Laboratory. Downloaded on April 01,2026 at 07:21:59 UTC from IEEE Xplore. Restrictions apply.